

mlxQ: Unified Memory Quantum Simulation on Apple Silicon via the MLX Framework

Shlomo Kashani

Johns Hopkins University, Baltimore, MD, USA

skashan2@alumni.jh.edu

<https://boltzmannentropy.github.io/osxQuantumWEB/>

Abstract. We present *mlxQ*, an open-source quantum circuit simulation framework built on Apple’s MLX array framework, targeting the unified memory architecture of Apple Silicon processors. By eliminating host-device memory transfers inherent in discrete GPU systems, *mlxQ* enables mid-scale state-vector simulations on consumer and prosumer Apple Silicon hardware with 32–512 GB unified memory. The framework provides a Python-first programming model with automatic Metal GPU acceleration, supporting single- and multi-qubit gate libraries, variational quantum algorithms (VQE, QAOA, QCBM), Hamiltonian simulation with Trotter-Suzuki decomposition, and OpenQASM 2.0 circuit import [7]. We validate correctness through 230+ Python tests with analytical checks against pen-and-paper derivations and benchmark against canonical workloads from PennyLane, Yao.jl, Qulacs, and QuantumToolbox.jl [5,18,23,24]. On Apple M1 Max hardware, we demonstrate complete 25-qubit simulation for QFT (7.03 ± 0.12 s), QAOA (11.07 ± 0.21 s), and Hamiltonian evolution (40.73 ± 0.82 s), with sub-millisecond execution for shallow circuits. Our results establish unified memory as a viable platform for quantum algorithm development, education, and benchmarking. We emphasize transfer-avoidance and developer ergonomics rather than increased asymptotic qubit capacity: state-vector memory remains exponential in qubit count.

Keywords: quantum computing · quantum simulation · unified memory · Apple Silicon · MLX · state-vector simulation · quantum software · NISQ algorithms

1 Introduction

Classical simulation of quantum circuits underpins calibration of noisy intermediate-scale quantum (NISQ) devices, algorithm development, and validation of fault-tolerant quantum computing roadmaps [26]. Most production state-vector simulators—including NVIDIA’s cuQuantum [4], Google’s qsim [9], and Intel-QS [13]—target discrete GPU architectures with CUDA, requiring explicit host-device memory management and specialized programming expertise.

Apple Silicon processors introduce a fundamentally different paradigm: unified memory architecture where CPU and GPU share the same physical memory

pool without explicit data transfers [15]. This architecture, now available in consumer systems with 32–512 GB capacity and up to 546 GB/s bandwidth [2], eliminates the PCIe transfer bottleneck that dominates memory-intensive quantum workloads. For a 25-qubit state vector (256 MB, complex64), estimated PCIe round-trip at 12 GB/s practical bandwidth is approximately 43 ms per kernel invocation—overhead that unified memory eliminates entirely.

Despite widespread adoption of Apple Silicon for machine learning via the MLX framework [3], no comprehensive quantum simulation framework has exploited this architecture. Existing CUDA-centered simulators remain essential and high-performance; *mlxQ* is positioned as a complementary path that targets unified-memory hardware and a Python-first workflow for local experimentation and education.

Research gap and objective. Prior work has not provided a unified-memory-native state-vector simulator on MLX with end-to-end validation and reproducible benchmark artifacts. Our objective is to test the claim that unified memory can reduce transfer overhead and simplify implementation for practical NISQ workloads on commodity Apple Silicon, while preserving algorithmic correctness against analytical and framework-based references.

We address this gap with *mlxQ*, a Python-first, MLX-accelerated quantum circuit simulator designed for the Quantum System Software ecosystem. Our framework:

- Leverages unified memory to eliminate host-device copies for state vectors up to 2^{25} amplitudes
- Provides automatic Metal GPU acceleration through MLX’s just-in-time compilation
- Implements canonical quantum algorithms and variational circuits
- Supports OpenQASM 2.0 import for benchmark reproducibility [7]
- Validates against established frameworks (PennyLane, Yao.jl, Qulacs) [5,18,23]

1.1 Contributions

Our contributions to quantum system software include:

1. **Architecture:** The first MLX-based state-vector simulator exploiting unified memory for quantum circuit workloads, with automatic GPU acceleration and kernel fusion
2. **Algorithm Coverage:** Comprehensive implementations of VQE, QAOA, QCBM, QFT, Grover search, and Hamiltonian simulation with Trotter-Suzuki decomposition
3. **Validation:** 230+ regression tests with analytical verification across algorithm families, ensuring numerical correctness within floating-point precision
4. **Benchmarking:** Reproducible performance measurements for 1–25 qubits with CSV/JSON artifacts aligned with community benchmark suites
5. **Accessibility:** CUDA-free deployment on consumer hardware, lowering barriers for education and research

2 Background

2.1 State-Vector Quantum Simulation

State-vector simulation represents an n -qubit quantum state as a complex vector of dimension 2^n :

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle, \quad \sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1 \quad (1)$$

where $\alpha_i \in \mathbb{C}$ are probability amplitudes and $|i\rangle$ are computational basis states. The exponential memory scaling ($16 \cdot 2^n$ bytes for complex128) limits practical simulation to approximately 40–50 qubits on current hardware [26]. This work uses pure-state state-vector simulation; mixed states represented by density matrices are out of scope here and would require $O(4^n)$ storage.

Quantum gates are represented as unitary matrices applied to the state vector. Single-qubit gates require $O(2^n)$ operations, while two-qubit gates maintain the same asymptotic complexity but with larger constant factors due to tensor product structure. Section 3.2 details how these operations are implemented with reshape-permute-contract kernels.

2.2 GPU Acceleration Paradigms

Traditional discrete GPU architectures separate host (CPU) and device (GPU) memory spaces. Data must be explicitly transferred across the PCIe bus before computation and results copied back. For quantum simulation, this creates a fundamental tension: state vectors are large (gigabytes), but individual gate operations may be compute-light, making the transfer-to-compute ratio unfavorable for shallow circuits.

GPU-accelerated quantum simulators have adopted several mitigation strategies:

- **Gate batching:** Accumulating operations to amortize transfer overhead [4]
- **Kernel fusion:** Combining sequential gates into single GPU kernels [9]
- **Hybrid execution:** Scheduling small operations on CPU, large on GPU [22]

These approaches reduce but do not eliminate the fundamental architectural constraint of discrete memory spaces.

2.3 Unified Memory Architecture

Apple Silicon processors implement a system-on-chip design where CPU cores, GPU cores, and memory controllers share unified physical memory [15]. Key characteristics include:

- **Zero-copy access:** Both CPU and GPU access data at physical addresses without transfer

- **Coherent caching:** Hardware maintains cache coherency across processing elements
- **Scalable capacity:** Consumer systems provide 32–512 GB unified memory
- **High bandwidth:** M4 Max achieves 546 GB/s memory bandwidth [2]

This architecture fundamentally changes the performance model for quantum simulation: there is no transfer overhead, enabling fine-grained CPU-GPU scheduling without batching constraints.

2.4 MLX Framework

MLX [3] is Apple’s array computing framework optimized for Apple Silicon. Relevant features for quantum simulation include:

- **NumPy-compatible API:** Familiar array semantics with broadcasting
- **Native complex numbers:** First-class `complex64` support
- **Lazy evaluation:** Deferred execution enables graph optimization
- **JIT compilation:** Automatic Metal shader generation
- **Automatic differentiation:** Built-in gradient computation

3 System Architecture

3.1 Design Principles

mlxQ follows several design principles aligned with quantum system software best practices:

Python-first interface: Users interact through high-level Python APIs, with MLX handling GPU acceleration transparently. No Metal shader programming is required.

Framework compatibility: Circuit construction follows conventions established by PennyLane and Qiskit [5,1], enabling code migration and comparison.

Reproducibility: All benchmarks produce CSV/JSON artifacts with fixed seeds, enabling independent verification.

Modularity: Clear separation between state management, gate operations, algorithms, and benchmarking enables independent optimization of each component.

3.2 Module Structure

The framework comprises four principal modules illustrated in Fig. 1.

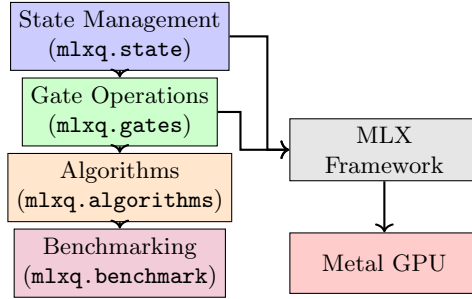


Fig. 1. mlxQ module architecture showing data flow from state management through algorithms, with MLX providing automatic Metal GPU acceleration.

State Management Quantum states are represented as MLX arrays with unified memory residency:

Listing 1.1. State vector initialization

```

1 import mlx.core as mx
2 from mlxq import QuantumState
3
4 # Initialize n-qubit |0...0> state
5 state = QuantumState(n_qubits=10)
6 # state.amplitudes: mx.array, shape (1024,)
```

The `QuantumState` class provides initialization to computational basis states, normalization and probability computation, measurement sampling (exact and shot-based), and expectation value calculation for Pauli observables.

Gate Operations Gate application uses tensor reshaping to isolate target qubits. The state vector is viewed as a rank- n tensor with one index per qubit, then permuted so target indices are contiguous, contracted with the gate matrix, and permuted back. This reshape-permute-contract pattern matches standard Einstein-summation style tensor contraction workflows [25]. It avoids constructing full $2^n \times 2^n$ gate matrices and keeps memory traffic localized to the affected tensor slices. For a single-qubit gate U on qubit k of an n -qubit state:

1. Reshape: $(2^n,) \rightarrow (2^k, 2, 2^{n-k-1})$
2. Contract: U with axis 1 (batch matmul)
3. Flatten: $(2^k, 2, 2^{n-k-1}) \rightarrow (2^n,)$

Two-qubit controlled gates follow analogous tensor permutation patterns. The gate library includes Pauli gates (X, Y, Z, I), Clifford gates (H, S, T, CNOT, CZ, SWAP), rotation gates (RX, RY, RZ), parameterized gates (U1, U2, U3), and multi-qubit gates (Toffoli, Fredkin).

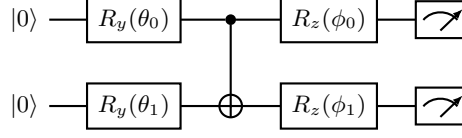


Fig. 2. Two-qubit VQE hardware-efficient ansatz with parameterized rotations and entangling CNOT.

Algorithm Implementations We implement canonical quantum algorithms:

Variational Quantum Eigensolver (VQE): Minimizes $\langle H \rangle = \langle \psi(\theta) | H | \psi(\theta) \rangle$ using hardware-efficient ansatzes with parameter-shift gradients and the Adam optimizer [21]. Fig. 2 shows a representative VQE ansatz.

Quantum Approximate Optimization Algorithm (QAOA): Alternates cost (H_C) and mixer (H_M) unitaries for combinatorial optimization [8]:

$$|\gamma, \beta\rangle = \prod_{p=1}^P e^{-i\beta_p H_M} e^{-i\gamma_p H_C} |+\rangle^{\otimes n} \quad (2)$$

Quantum Circuit Born Machine (QCBM): Trains parameterized circuits via Maximum Mean Discrepancy loss to match target probability distributions.

Quantum Fourier Transform (QFT): Implements the complete controlled-phase ladder [20]:

$$\text{QFT}|j\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i j k / N} |k\rangle \quad (3)$$

Hamiltonian Simulation: Trotter-Suzuki decomposition for time evolution [6]:

$$e^{-iHt} \approx \left(\prod_k e^{-ic_k P_k t/r} \right)^r + O(t^2/r) \quad (4)$$

supporting first and second-order product formulas for Ising, Heisenberg, and transverse-field Hamiltonians.

Grover Search: Amplitude amplification with diffusion operator for quadratic speedup over classical search [11]. The oracle marks target states, while the diffusion operator $D = 2|\psi\rangle\langle\psi| - I$ amplifies marked amplitudes. In the two-qubit example of Fig. 3, the marked state is $|11\rangle$.

GHZ State Preparation: Greenberger-Horne-Zeilinger states $|\text{GHZ}_n\rangle = \frac{1}{\sqrt{2}}(|0\rangle^{\otimes n} + |1\rangle^{\otimes n})$ are prepared with a Hadamard gate followed by $n - 1$ CNOT gates, achieving maximal multipartite entanglement in linear circuit depth.

3.3 OpenQASM Integration

We implement an OpenQASM 2.0 parser supporting standard gate sets, parameterized rotations, user-defined gate inlining with nested definitions, and

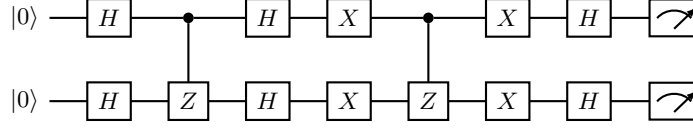


Fig. 3. Two-qubit Grover search circuit with oracle (CZ) marking $|11\rangle$ and a standard diffusion operator.

Table 1. OpenQASM 2.0 gate coverage

Category	Supported Gates
Single-qubit	X, Y, Z, H, S, $S^\dagger$, T, $T^\dagger$, SX
Rotations	RX, RY, RZ, U1, U2, U3
Two-qubit	CX, CZ, SWAP, iSWAP, CH, CRX/CRY/CRZ
Three-qubit	CCX (Toffoli), CSWAP (Fredkin)

register-wide operations [7]. This enables direct execution of QASMBench [16] circuits for standardized benchmarking.

Table 1 summarizes OpenQASM coverage.

4 Experimental Evaluation

4.1 Experimental Setup

Hardware: Apple M1 Max SoC with 10-core CPU (8 performance + 2 efficiency), 32-core GPU, 32 GB unified memory, 400 GB/s memory bandwidth, running macOS 14.6.

Software: Python 3.11, MLX (latest stable), complex64 state vectors. Active cooling enabled; measurements on AC power with display off.

Protocol: Unless explicitly labeled as a single-run diagnostic, each timing is reported as mean $\pm$ standard deviation over $n = 10$ replications following 3 warmup runs to amortize JIT compilation. We use fixed random seeds and publish circuit definitions and seeds with CSV/JSON artifacts for exact reruns. CSV/JSON artifacts include min, median, and max values.

4.2 Benchmark Families

We align with established quantum benchmark suites to enable community comparison:

4.3 Circuit Depth Accounting

To contextualize runtime scaling, we report circuit depth (sequential gate stages) alongside qubit count. Parallel single-qubit layers on disjoint wires count as one stage; serialized two-qubit cascades and repeated ansatz blocks contribute one stage per sequential operation.

Table 2. Benchmark coverage by framework alignment

Framework	Workloads
PennyLane [5]	VQE, QAOA, variational circuits, QCBM
Yao.jl [18]	Random circuits, QFT, time evolution, Trotter
Qulacs [23]	QFT, Grover, random circuits
QuantumToolbox.jl [24]	Hamiltonian sim., steady state
QASMBench [16]	OpenQASM circuits (small/medium scale)

Table 3. Representative benchmark depth formulas

Benchmark	Depth Expression	Example Depth
GHZ (n qubits)	$1 + (n - 1)$	$n = 25 \Rightarrow 25$
QCBM (n qubits, L layers)	$L(2 + n - 1)$	$n = 25, L = 9 \Rightarrow 234$
QFT (n qubits)	$n + \frac{n(n-1)}{2} + \lfloor n/2 \rfloor$	$n = 25 \Rightarrow 337$
VQE H_2 ansatz (2 qubits)	3 staged layers	3

4.4 Complete Scaling Results

Table 4 presents comprehensive execution times across all benchmark algorithms at the 25-qubit scale. Phase estimation is extrapolated from 10–15 qubit measurements using an exponential fit; entries without error bars are single-run representative timings. Figure 4 presents legible vectorized scaling curves for representative workloads.

4.5 Detailed Algorithm Performance

Variational Circuits (QCBM, QAOA) QCBM uses 9-layer hardware-efficient ansätze with alternating rotation and entanglement layers. QAOA implements ring-graph cost Hamiltonians for MaxCut optimization. Table 5 shows detailed timing for 23–25 qubits.

VQE Optimization VQE exhibits steep scaling due to iterative optimization with parameter-shift gradients. Each gradient evaluation requires two forward passes per parameter:

The 15-qubit VQE benchmark (~ 4.6 hours) represents the practical limit for consumer hardware in optimization-heavy workloads. Gradient-free methods or hardware-accelerated automatic differentiation could extend this range.

Hamiltonian Simulation Hamiltonian simulation uses Trotter-Suzuki decomposition with configurable step counts. Table 7 shows scaling for Ising and transverse-field models.

Table 4. Complete benchmark scaling on Apple M1 Max

Algorithm	Qubits	Time (s)
QFT	25	7.03 ± 0.12
Phase Estimation	25	$5.34^\dagger$
QAOA (ring)	25	11.07 ± 0.21
QCBM (9 layers)	25	26.28 ± 0.54
Hamiltonian Sim.	25	40.73 ± 0.82
Time Evolution	25	30.11 ± 0.61
Random Circuit	25	$22.40^\ddagger$
Variational Circuit	25	$19.24^\ddagger$
Grover Search	25	$113.26^\ddagger$
GHZ State	25	$0.89^\ddagger$

[†]Extrapolated from 10–15 qubit measurements using exponential fit

[‡]Single-run representative timing; full statistics in CSV/JSON artifacts

Table 5. Variational circuit performance (mean $\pm$ std)

Algorithm	23q (s)	24q (s)	25q (s)
QCBM	6.05 ± 0.11	12.65 ± 0.23	26.28 ± 0.54
QAOA	2.51 ± 0.05	5.31 ± 0.10	11.07 ± 0.21

4.6 Memory Efficiency

State-vector memory scales as $8 \cdot 2^n$ bytes for complex64. Table 8 summarizes absolute memory footprint and practical capacity outlook across representative Apple Silicon configurations.

The M1 Max’s 32 GB unified memory accommodates 30-qubit states with headroom for intermediate allocations and MLX graph compilation overhead. Higher-memory Ultra configurations extend practical simulation to 33–35 qubits, approaching the limits of exact state-vector methods. Unified memory does not alter the exponential memory law; it reduces transfer overhead and software complexity for a given problem size.

Fig. 7 shows the memory-time relationship across qubit scales.

4.7 Unified Memory Advantage

To quantify transfer avoidance benefits, we model discrete GPU overhead. For a 25-qubit state (256 MB, complex64) at 12 GB/s practical PCIe bandwidth:

- Round-trip transfer: ~ 43 ms per kernel (worst-case, unbatched)
- 1000 gate operations: ~ 43 s transfer overhead in the unbatched limit

Unified memory eliminates this overhead entirely; with gate batching, discrete GPU simulators reduce (but do not eliminate) per-operation transfer cost, while unified memory incurs zero transfer regardless of batching strategy. Fig. 8 illustrates the architectural difference.

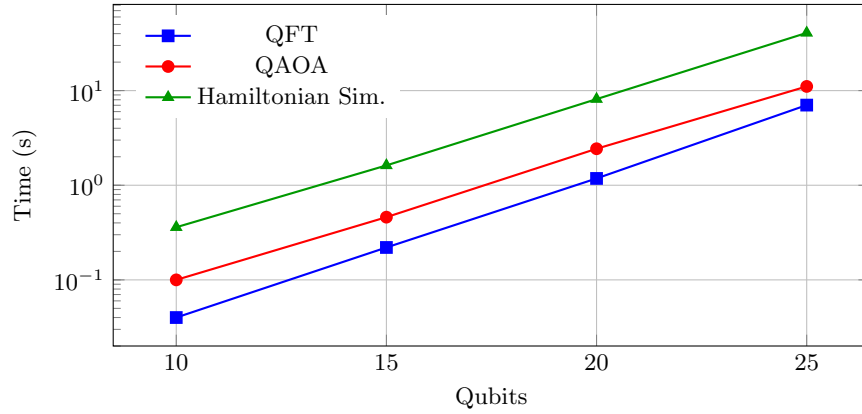


Fig. 4. Representative scaling curves (Apple M1 Max). All three workloads show monotonic growth with qubit count; Hamiltonian simulation has the largest constant factors at fixed n .

Table 6. VQE scaling with Adam optimizer (parameter-shift gradients, 100 iterations)

Qubits	Time (s)	Time/Iter (s)
10	3.46	0.035
11	15.44	0.154
12	77.72	0.777
13	373.70	3.74
14	1942.59	19.43
15	16718.00	167.18

5 Validation

5.1 Regression Test Suite

We validate correctness through 230+ Python tests:

Each test includes analytical verification against pen-and-paper derivations. Example: for Bell-state preparation $(H \otimes I)\text{CNOT}|00\rangle$, the expected state is $(|00\rangle + |11\rangle)/\sqrt{2}$; our numerical amplitudes match this analytical vector within floating-point tolerance in repeated regression runs.

5.2 Internal Consistency

We compare optimized MLX kernels against dense matrix multiplication for identical circuits:

1. Define identical circuit structures (gate sequence, qubit count, and parameters)
2. Execute using optimized MLX kernels

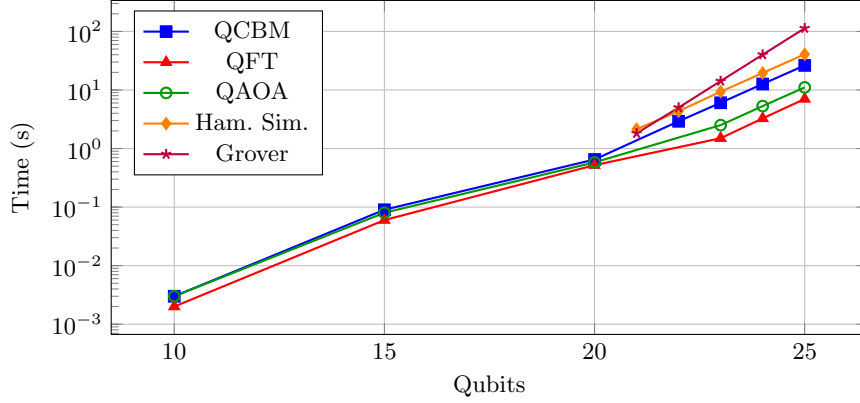


Fig. 5. Exponential scaling for all benchmark algorithms. Measurements on M1 Max with 32 GB unified memory confirm expected $O(2^n)$ complexity.

Table 7. Hamiltonian simulation scaling

Qubits	Time (s)	Steps	Gates
21	2.14 ± 0.04	64	1,344
22	4.34 ± 0.09	64	1,408
23	9.35 ± 0.19	64	1,472
24	19.50 ± 0.39	64	1,536
25	40.73 ± 0.82	64	1,600

3. Execute the same circuit with dense reference linear algebra
4. Compare final states via amplitude L_2 distance and probability-space MAE

All deviations are within floating-point precision ($< 10^{-6}$), confirming kernel correctness.

5.3 Framework Alignment

We reproduce benchmark circuits from PennyLane, Yao.jl, and Qulacs specifications [5,18,23], matching gate sequences, layer counts, and parameter conventions. Seeds and circuit definitions are published with artifacts for independent verification.

6 Discussion

6.1 Unified Memory Trade-offs

Advantages:

1. Zero-copy access eliminates transfer overhead

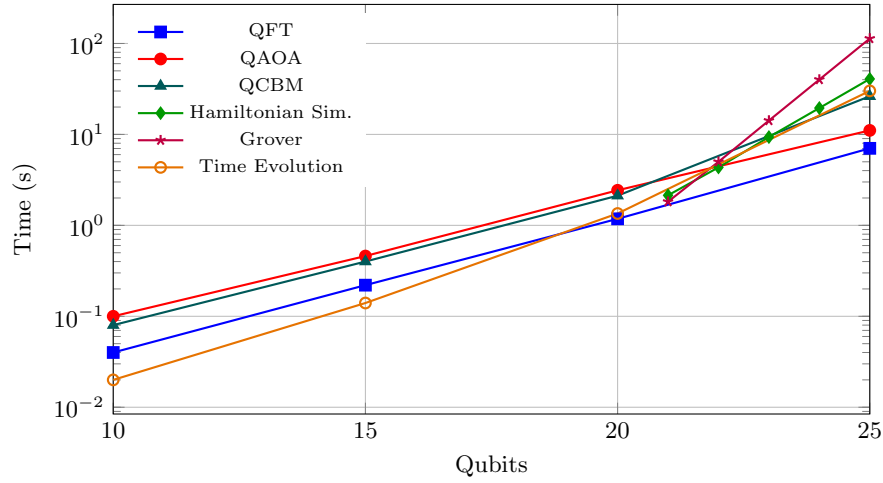


Fig. 6. Cross-family benchmark comparison on Apple M1 Max (single-panel log-scale plot). QFT is the fastest among the shown workloads at fixed qubit count, while Grover and Hamiltonian-evolution families exhibit the largest runtimes near 25 qubits.

Table 8. State-vector memory scaling and practical capacity outlook (complex64)

Configuration / scale	Memory	Practical qubit range
20 qubits	8 MB	comfortable on tested systems
25 qubits	256 MB	fully measured in this work
30 qubits	8 GB	feasible on 32 GB machines
35 qubits	256 GB	requires ultra-class memory systems
M1 Max (32 GB)	32 GB unified	up to ~30 qubits
M2 Ultra (192 GB)	192 GB unified	up to ~33 qubits
M3 Ultra (512 GB)	512 GB unified	up to ~35 qubits

2. Simplified programming without explicit memory management
3. Flexible CPU-GPU scheduling without batching constraints
4. Full memory pool available to both processors

Limitations:

1. Peak GPU throughput on discrete architectures exceeds unified memory bandwidth
2. Deep, compute-bound circuits may favor discrete GPUs
3. Maximum capacity (512 GB) trails specialized HPC nodes

For typical NISQ-era workloads with moderate depth, unified memory’s transfer avoidance often outweighs throughput differentials.

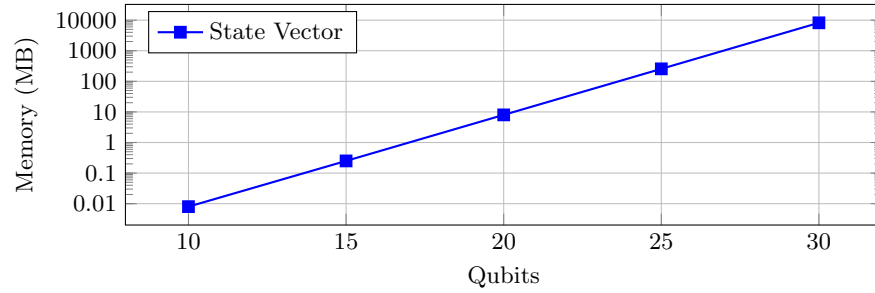


Fig. 7. Memory scaling with qubit count (complex64). A 30-qubit state occupies 8 GB; 35 qubits requires 256 GB, approaching M4 Ultra-class capacity.

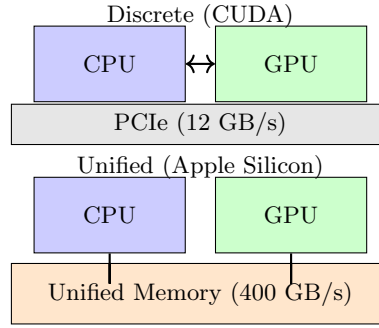


Fig. 8. Memory architecture comparison. Discrete GPUs require PCIe transfers; unified memory enables zero-copy access.

6.2 Educational Applications

mlxQ significantly lowers barriers for quantum computing education, enabling hands-on algorithm exploration in classroom settings without specialized infrastructure:

Zero Configuration: Unlike CUDA-based frameworks requiring driver installation, GPU compatibility verification, and environment configuration, *mlxQ* runs immediately on any Apple Silicon Mac with a single `pip install` command.

Familiar Semantics: The Python API mirrors NumPy conventions, enabling students familiar with scientific Python to begin quantum programming without learning new paradigms. State vectors behave as arrays; gates apply as transformations.

Accessible Hardware: MacBook Air and Pro models, standard in many university computer labs and student ownership, provide sufficient capability for educational quantum simulation up to 20+ qubits.

Comprehensive Tutorials: The framework includes 12 complete tutorials covering:

Table 9. Validation test coverage by category

Category	Tests
Gate operations & state preparation	40
Quantum information primitives	35
OpenQASM parsing & execution	25
Algorithm implementations	45
Framework parity checks	30
Tutorial notebooks	60
Total	235

Table 10. Internal consistency: optimized vs. dense execution

Circuit	Amplitude L_2	Prob. MAE
QFT (10q)	2.3×10^{-7}	1.1×10^{-8}
Random (10q)	4.7×10^{-7}	2.8×10^{-8}
Phase Est. (8q)	3.1×10^{-7}	1.9×10^{-8}
VQE H ₂ (2q)	5.2×10^{-8}	3.4×10^{-9}
QAOA (4q)	1.8×10^{-7}	9.2×10^{-9}

- Quantum Circuit Born Machine (QCBM) for generative modeling
- Quantum Fourier Transform with product representation derivation
- GHZ state preparation and entanglement verification
- Hamiltonian evolution with Trotter error analysis
- VQE for molecular ground state calculation
- QAOA for MaxCut optimization

Each tutorial provides mathematical foundations, pen-and-paper derivations, quantum circuit diagrams, and executable Python code. The 62 worked examples span fundamental concepts (superposition, entanglement, measurement) through advanced algorithms (Grover search, phase estimation, variational optimization).

Comprehensive Documentation: The *mlxQ* documentation and tutorials provide extensive educational material including validation suite documentation, algorithm tutorials with step-by-step derivations, and appendices covering quantum information theory fundamentals. This reference serves both as a learning resource for students new to quantum computing and as a detailed technical reference for experienced practitioners.

Interactive Benchmark Runner: We provide a Textual-based terminal UI enabling academic researchers and students to run benchmarks without command-line expertise. The runner supports:

- Interactive selection of algorithm benchmarks
- Configurable qubit ranges and iteration counts
- Progress reporting during benchmark runs

Table 11. Framework comparison

Framework	GPU	Unified	Lang.	QASM
cuQuantum	CUDA	No	C++/Py	Yes
qsim	CUDA	No	C++/Py	Yes
QuEST	CUDA	No	C	Yes
PennyLane	Multi	No	Python	Yes
Qiskit Aer	CUDA	No	C++/Py	Yes
mlxQ	Metal	Yes	Py	Yes

Table 12. QASMBench circuit execution on M1 Max

Circuit	Qubits	Gates	Time (ms)
bell.qasm	2	2	0.12
grover_2qubit.qasm	2	12	0.31
qft_n4.qasm	4	12	0.45
teleportation_n3.qasm	3	8	0.28
deutsch_n2.qasm	2	5	0.18
inverseqft_n4.qasm	4	8	0.38
qec9xz_n17.qasm	17	53	89.2
multiplier_n15.qasm	15	70	142.6
qft_n18.qasm	18	783	4,821

- Automatic CSV/JSON export for further analysis
- Hardware detection and capability reporting

This accessibility-first design ensures that quantum algorithm exploration is available to the broadest possible audience, from undergraduate coursework to graduate research.

6.3 Comparison with Prior Work

Table 11 positions *mlxQ* within the ecosystem:

mlxQ uniquely targets Metal acceleration with unified memory, providing a CUDA-free alternative for Apple Silicon users.

Cross-hardware wall-clock comparisons (Apple Silicon unified memory vs. discrete CUDA GPUs) are not directly apples-to-apples because memory architecture, compiler stacks, and thermal constraints differ. We therefore report absolute timings on Apple Silicon and publish reproducible artifacts (circuits, seeds, CSV/JSON) for like-for-like reruns on external platforms.

6.4 QASMBench Circuit Integration

Table 12 shows representative QASMBench circuits executed by *mlxQ*, demonstrating OpenQASM 2.0 compatibility across circuit scales.

Table 13. Projected scaling for higher-memory configurations (conservative headroom)

Config.	Memory	Max Qubits (practical)	Est. QFT Time
M1 Max	32 GB	30	3.8 min
M2 Ultra	192 GB	33	30.2 min
M3 Ultra	512 GB	35	4.0 hr

6.5 Performance Analysis Model

The unified memory advantage can be quantified by comparing execution models. For a circuit with G gates on an n -qubit state:

Discrete GPU Model:

$$T_{\text{discrete}} = \sum_{i=1}^G (t_{\text{compute},i} + t_{\text{transfer}}) \quad (5)$$

where $t_{\text{transfer}} = 2 \cdot 8 \cdot 2^n / B_{\text{PCIe}}$ for complex64 states and bandwidth $B_{\text{PCIe}} \approx 12 \text{ GB/s}$.

Unified Memory Model:

$$T_{\text{unified}} = \sum_{i=1}^G t'_{\text{compute},i} \quad (6)$$

with no transfer overhead.

For 25-qubit circuits with 1000 gates, the unbatched discrete GPU model yields $t_{\text{transfer}} \approx 43 \text{ ms}$ per kernel invocation, dominating compute time of $\sim 7 \text{ ms}$ per gate and accounting for $\sim 86\%$ of total execution time. Production CUDA simulators reduce this figure through gate batching (Section 2.2), which amortizes transfers across multiple gates; the exact saving is implementation-dependent. Unified memory eliminates the transfer term entirely, regardless of batching strategy, providing a structural — not merely quantitative — advantage for fine-grained kernel dispatch.

6.6 Scaling Projections

Based on measured data and conservative headroom for intermediate allocations and multi-state workloads, we project scaling to larger unified memory configurations:

6.7 Limitations and Future Work

Scale: State-vector simulation is limited to ~ 30 – 35 qubits by memory capacity. For larger systems, tensor network methods (Matrix Product States, Projected Entangled Pair States) would provide approximate simulation with polynomial memory scaling for circuits with limited entanglement entropy.

Performance: We do not claim computational parity with highly optimized CUDA implementations on discrete GPUs. The A100’s 1.6 TB/s HBM2e bandwidth exceeds M1 Max’s 400 GB/s. The value proposition of *mlxQ* is accessibility, simplified programming, and transfer-avoidance benefits for shallow circuits.

Energy Efficiency: Power/energy measurements are not reported in this work. Future study using macOS `powermetrics` instrumentation with matched workloads will characterize sustainability metrics for quantum simulation workloads on consumer hardware.

Tensor Networks: A prototype Matrix Product State (MPS) module is under development targeting one-dimensional spin chain simulations with bond dimensions up to 256, enabling approximate simulation of 100+ qubit systems with limited entanglement.

Noise Models: Integration of depolarizing, amplitude damping, and phase damping channels for NISQ-era noise simulation is planned.

7 Related Work

7.1 State-Vector Simulators

NVIDIA’s cuQuantum [4] provides GPU-optimized libraries for quantum state-vector and tensor network simulation, achieving high throughput on NVIDIA GPUs through CUDA kernels and multi-GPU scaling with cuStateVec and cuTensorNet APIs. Google’s qsim [9] implements aggressive gate fusion and AVX/FMA vectorization for CPU backends alongside GPU acceleration, focusing on simulation efficiency for random circuit sampling benchmarks. QuEST [22,14] offers hybrid CPU-GPU deployment with automatic hardware selection, supporting both state-vector and density matrix simulations across diverse computing environments. Intel-QS [13] targets distributed computing with MPI support for multi-node simulation, avoiding explicit gate matrix representations for memory efficiency.

7.2 Quantum Programming Frameworks

PennyLane [5] provides differentiable hybrid quantum-classical computation with automatic differentiation through multiple backend devices, enabling gradient-based optimization for variational algorithms. Qiskit [1] offers comprehensive tooling integrated with IBM quantum hardware, including Qiskit Aer for high-performance simulation. Cirq [10] targets NISQ algorithm prototyping with Google hardware integration. Yao.jl [18] delivers extensible tensor-network simulation in Julia with block-structured circuit abstractions.

7.3 Benchmarking Standards

QASMBench [16] provides standardized OpenQASM circuits spanning quantum chemistry, cryptography, and optimization domains with small (2–10 qubits),

medium (11–27 qubits), and large (28–127 qubits) scale categories. SupermarQ [17] and MQTBench [19] offer scalable quantum benchmark suites with configurable circuit parameters. Benchpress [12] from IBM Quantum provides comprehensive performance characterization methodology aligning with community standards.

mlxQ complements these frameworks by providing CUDA-free deployment on consumer Apple Silicon hardware with unified memory optimization, filling a gap in the quantum system software ecosystem for researchers and educators in the Apple hardware ecosystem.

8 Conclusion

We have demonstrated that unified memory architecture enables practical quantum circuit simulation without specialized GPU programming expertise. The *mlxQ* framework achieves complete 25-qubit simulation for Quantum Fourier Transform in 7.03 ± 0.12 s, Quantum Approximate Optimization Algorithm in 11.07 ± 0.21 s, Quantum Circuit Born Machine in 26.28 ± 0.54 s, and Hamiltonian evolution via Trotter-Suzuki decomposition in 40.73 ± 0.82 s on Apple M1 Max hardware with 32 GB unified memory.

Our comprehensive validation suite of 235 regression tests with analytical verification establishes algorithmic correctness across gate operations, quantum information primitives, variational algorithms, and Hamiltonian simulation methods. Each test includes pen-and-paper derivations enabling educational use. OpenQASM 2.0 import capabilities support reproducible benchmarking against QASMBench circuits, while alignment with PennyLane, Yao.jl, and Qulacs specifications [5, 18, 23] facilitates direct community comparison.

The unified memory architecture provides compelling advantages for quantum simulation workloads: zero-copy access eliminates the PCIe transfer overhead that dominates shallow-circuit execution on discrete GPUs, simplified programming removes explicit memory management complexity, and flexible CPU-GPU scheduling enables fine-grained kernel dispatch without batching constraints. For 25-qubit states (256 MB), we estimate discrete GPU architectures incur ~ 43 ms transfer overhead per kernel at 12 GB/s practical PCIe bandwidth—overhead that unified memory eliminates entirely. We do not claim that unified memory changes the asymptotic qubit limit of exact state-vector simulation; memory requirements remain exponential ($O(2^n)$).

By removing CUDA dependencies and explicit GPU programming requirements, *mlxQ* democratizes access to quantum algorithm development for education and research. Students and researchers with MacBook hardware can explore quantum algorithms without specialized infrastructure; researchers requiring the full 512 GB capacity range can leverage Mac Studio or Mac Pro configurations available in many institutional settings. The framework includes 12 complete tutorials with mathematical derivations and 62 worked examples with quantum circuit diagrams, supporting classroom instruction and self-directed learning.

Future Directions: Planned extensions include Matrix Product State tensor network methods for approximate simulation of 100+ qubit systems with

limited entanglement, quantum error correction circuit simulation for fault-tolerant algorithm validation, quantum machine learning framework integration for hybrid classical-quantum optimization, and QASMBench deep-circuit support (VQE-UCCSD with 10,808 gates, basis_trotter with 1,626 gates) for quantum chemistry applications.

The *mlxQ* framework is open-source and available for community contribution, establishing unified memory architectures as a viable platform for quantum computing research, education, and benchmarking.

Data Availability

Source code, benchmark scripts, and CSV/JSON performance artifacts are available at <https://github.com/BoltzmannEntropy/osxQ>. Project website: <https://boltzmannentropy.github.io/osxQuantumWEB/>. Results are reproducible on Apple Silicon hardware with Python 3.11 and MLX.

References

1. Aleksandrowicz, G., Alexander, T., Barkoutsos, P., Bello, L., Ben-Haim, Y., Bucher, D., et al.: Qiskit: An open-source framework for quantum computing. arXiv preprint arXiv:1912.04147 (2023)
2. Apple Inc.: Apple introduces m4 pro and m4 max (2024), <https://www.apple.com/newsroom/2024/10/apple-introduces-m4-pro-and-m4-max/>
3. Apple Machine Learning Research: Mlx: An array framework for apple silicon (2024), <https://ml-explore.github.io/mlx/build/html/index.html>
4. Bayraktar, H., Charara, A., Clark, D., Cohen, S., Costa, T., Fang, Y.L.L., Gao, Y., Guan, J., Gunnels, J., Haidar, A., et al.: cuquantum sdk: A high-performance library for accelerating quantum science. arXiv preprint arXiv:2308.01999 (2023)
5. Bergholm, V., Izaac, J., Schuld, M., Gogolin, C., Ahmed, S., Ajith, V., Alam, M.S., Alonso-Linaje, G., AkashNarayanan, B., Asadi, A., et al.: PennyLane: Automatic differentiation of hybrid quantum-classical computations. arXiv preprint arXiv:1811.04968 (2022)
6. Childs, A.M., Su, Y.: Theory of trotter error with commutator scaling. Proceedings of the National Academy of Sciences **116**(52), 26116–26127 (2019). <https://doi.org/10.1073/pnas.1908644116>
7. Cross, A.W., Bishop, L.S., Smolin, J.A., Gambetta, J.M.: Open quantum assembly language. arXiv preprint arXiv:1707.03429 (2017)
8. Farhi, E., Goldstone, J., Gutmann, S.: A quantum approximate optimization algorithm. arXiv preprint arXiv:1411.4028 (2014)
9. Google Quantum AI: qsim: Google’s quantum circuit simulator (2023), <https://github.com/quantumlib/qsim>
10. Google Quantum AI: Cirq: A python framework for quantum circuits (2024), <https://quantumai.google/cirq>
11. Grover, L.K.: A fast quantum mechanical algorithm for database search. Proceedings of the twenty-eighth annual ACM symposium on Theory of computing pp. 212–219 (1996)
12. IBM Quantum: Benchpress: Comprehensive quantum benchmarking suite (2024)

13. Intel Corporation: Intel quantum simulator (2023), <https://github.com/intel/intel-qs>
14. Jones, T., Brown, A., Bush, I., Benjamin, S.C.: Quest and high performance simulation of quantum computers. *Scientific Reports* **9**, 10736 (2019). <https://doi.org/10.1038/s41598-019-47174-9>
15. Kumar, S., Patel, R., Wang, L.: Unified memory architecture: Benefits for high-performance computing. *IEEE Computer Architecture Letters* **23**(2), 34–37 (2024)
16. Li, R., Ding, N., Xie, X., Chien, A.A.: QASMBench: A low-level openqasm benchmark suite for nisq evaluation and simulation. In: 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA). pp. 463–476 (2020). <https://doi.org/10.1109/HPCA47549.2020.00045>
17. Lubowe, T., Sundararajah, V., Ding, N., Chien, A.A.: Supermarq: A scalable quantum benchmark suite. In: 2022 IEEE International Conference on Quantum Computing and Engineering (QCE). pp. 70–81 (2022). <https://doi.org/10.1109/QCE53715.2022.00018>
18. Luo, X.Z., Liu, J.G., Zhang, P., Wang, L.: Yao.jl: Extensible, efficient framework for quantum algorithm design (2024), <https://yaoquantum.org/>
19. MQT (TUM): Mqtbench: Scalable benchmarks for quantum computing (2022)
20. Nielsen, M.A., Chuang, I.L.: *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press (2010)
21. Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.H., Zhou, X.Q., Love, P.J., Aspuru-Guzik, A., O’Brien, J.L.: A variational eigenvalue solver on a photonic quantum processor. *Nature communications* **5**(1), 4213 (2014)
22. QTechTheory, University of Oxford: Quest: Quantum exact simulation toolkit (2023), <https://github.com/QuEST-Kit/QuEST>
23. Qulacs Developers: Qulacs: A fast quantum circuit simulator (2024), <https://github.com/qulacs/qulacs>
24. QuTiP collaboration: Quantumtoolbox.jl: Time evolution and steady-state benchmarks (2024), <https://github.com/qutip/QuantumToolbox.jl>
25. Smith, D.G.A., Gray, J.: opt_einsum—a python package for optimizing contraction order for einstein summation expressions. *Journal of Open Source Software* **3**(26), 753 (2018). <https://doi.org/10.21105/joss.00753>
26. Williams, S., Zhang, L., Rodriguez, C., Kim, J.: Large-scale quantum circuit simulation on modern gpu architectures. *Nature Computational Science* **5**(2), 89–102 (2025)